JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class ClassLoader

java.lang.Object
    java.lang.ClassLoader

Direct Known Subclasses:

SecureClassLoader

```
public abstract class ClassLoader
extends Object
```

A class loader is an object that is responsible for loading classes. The class `ClassLoader` is an abstract class. Given the binary name of a class, a class loader should attempt to locate or generate data that constitutes a definition for the class. A typical strategy is to transform the name into a file name and then read a "class file" of that name from a file system.

Every `Class` object contains a `reference` to the `ClassLoader` that defined it.

`Class` objects for array classes are not created by class loaders, but are created automatically as required by the Java runtime. The class loader for an array class, as returned by `Class.getClassLoader()` is the same as the class loader for its element type; if the element type is a primitive type, then the array class has no class loader.

Applications implement subclasses of `ClassLoader` in order to extend the manner in which the Java virtual machine dynamically loads classes.

Class loaders may typically be used by security managers to indicate security domains.

The `ClassLoader` class uses a delegation model to search for classes and resources. Each instance of `ClassLoader` has an associated parent class loader. When requested to find a class or resource, a `ClassLoader` instance will delegate the search for the class or resource to its parent class loader before attempting to

find the class or resource itself. The virtual machine's built-in class loader, called the "bootstrap class loader", does not itself have a parent but may serve as the parent of a `ClassLoader` instance.

Class loaders that support concurrent loading of classes are known as *parallel capable* class loaders and are required to register themselves at their class initialization time by invoking the `ClassLoader.registerAsParallelCapable` method. Note that the `ClassLoader` class is registered as parallel capable by default. However, its subclasses still need to register themselves if they are parallel capable.
In environments in which the delegation model is not strictly hierarchical, class loaders need to be parallel capable, otherwise class loading can lead to deadlocks because the loader lock is held for the duration of the class loading process (see `loadClass` methods).

Normally, the Java virtual machine loads classes from the local file system in a platform-dependent manner. For example, on UNIX systems, the virtual machine loads classes from the directory defined by the `CLASSPATH` environment variable.

However, some classes may not originate from a file; they may originate from other sources, such as the network, or they could be constructed by an application. The method `defineClass` converts an array of bytes into an instance of class `Class`. Instances of this newly defined class can be created using `Class.newInstance`.

The methods and constructors of objects created by a class loader may reference other classes. To determine the class(es) referred to, the Java virtual machine invokes the `loadClass` method of the class loader that originally created the class.

For example, an application could create a network class loader to download class files from a server. Sample code might look like:

```
ClassLoader loader = new NetworkClassLoader(host, port);
Object main = loader.loadClass("Main", true).newInstance();
     . . .
```

The network class loader subclass must define the methods `findClass` and `loadClassData` to load a class from the network. Once it has downloaded the bytes that make up the class, it should use the method `defineClass` to create a class instance. A sample implementation is:

```
class NetworkClassLoader extends ClassLoader {
    String host;
    int port;

    public Class findClass(String name) {
        byte[] b = loadClassData(name);
        return defineClass(name, b, 0, b.length);
    }
```

```
        private byte[] loadClassData(String name) {
            // load the class data from the connection
             . . .
        }
    }
```

Binary names

Any class name provided as a `String` parameter to methods in `ClassLoader` must be a binary name as defined by *The Java™ Language Specification*.

Examples of valid class names include:

```
"java.lang.String"
"javax.swing.JSpinner$DefaultEditor"
"java.security.KeyStore$Builder$FileBuilder$1"
"java.net.URLClassLoader$3$1"
```

Since:

1.0

See Also:

`resolveClass(Class)`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**ClassLoader**()	Creates a new class loader using the ClassLoader returned by the method **getSystemClassLoader()** as the parent class loader.
protected	**ClassLoader**(**ClassLoader** parent)	Creates a new class loader using the specified parent class loader for delegation.

Method Summary

Deprecat

Modifier and Type	Method	Descrip
void	**clearAssertionStatus**()	Sets the asserti class lo and dis packag class as settings with the
protected **Class**<?>	**defineClass**(byte[] b, int off, int len)	**Deprec** Replace **define byte[]**
protected **Class**<?>	**defineClass**(**String** name, byte[] b, int off, int len)	Conver bytes in of class
protected **Class**<?>	**defineClass**(**String** name, byte[] b, int off, int len, **ProtectionDomain** protectionDomain)	Conver bytes in of class optiona Protec
protected **Class**<?>	**defineClass**(**String** name, **ByteBuffer** b, **ProtectionDomain** protectionDomain)	Conver into an class Cl optiona Protec
protected **Package**	**definePackage**(**String** name, **String** specTitle, **String** specVersion, **String** specVendor, **String** implTitle, **String** implVersion, **String** implVendor, **URL** sealBase)	Defines name in ClassL
protected **Class**<?>	**findClass**(**String** name)	Finds th specifie
protected **String**	**findLibrary**(**String** libname)	Returns path na library.
protected **Class**<?>	**findLoadedClass**(**String** name)	Returns the give if this l recorde virtual initiatin

		class w **name**.
protected **URL**	**findResource**(**String** name)	Finds t the give
protected **Enumeration**<**URL**>	**findResources**(**String** name)	Returns enumer objects the res given n
protected **Class**<?>	**findSystemClass**(**String** name)	Finds a specifie loading
protected **Object**	**getClassLoadingLock**(**String** className)	Returns for clas operati
protected **Package**	**getPackage**(**String** name)	Returns has bee this cla of its a
protected **Package**[]	**getPackages**()	Returns Packag this cla ancesto
ClassLoader	**getParent**()	Returns class lo delegat
URL	**getResource**(**String** name)	Finds t the give
InputStream	**getResourceAsStream**(**String** name)	Returns stream specifie
Enumeration<**URL**>	**getResources**(**String** name)	Finds a with th
static **ClassLoader**	**getSystemClassLoader**()	Returns class lo delegat
static **URL**	**getSystemResource**(**String** name)	Find a r specifie

		the sea load cla
static **InputStream**	**getSystemResourceAsStream**(**String** name)	Open fo resourc specifie the sea load cla
static **Enumeration**<**URL**>	**getSystemResources**(**String** name)	Finds a the spe from th used to
Class<?>	**loadClass**(**String** name)	Loads t the spe **name**.
protected **Class**<?>	**loadClass**(**String** name, boolean resolve)	Loads t the spe **name**.
protected static boolean	**registerAsParallelCapable**()	Registe parallel
protected void	**resolveClass**(**Class**<?> c)	Links th class.
void	**setClassAssertionStatus**(**String** className, boolean enabled)	Sets the asserti named in this any nes contain
void	**setDefaultAssertionStatus**(boolean enabled)	Sets the asserti class lo
void	**setPackageAssertionStatus**(**String** packageName, boolean enabled)	Sets the default for the
protected void	**setSigners**(**Class**<?> c, **Object**[] signers)	Sets the class.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ClassLoader

```
protected ClassLoader(ClassLoader parent)
```

Creates a new class loader using the specified parent class loader for delegation.

If there is a security manager, its `checkCreateClassLoader` method is invoked. This may result in a security exception.

Parameters:

`parent - The parent class loader`

Throws:

`SecurityException - If a security manager exists and its checkCreateClassLoader method doesn't allow creation of a new class loader.`

Since:

`1.2`

ClassLoader

```
protected ClassLoader()
```

Creates a new class loader using the `ClassLoader` returned by the method `getSystemClassLoader()` as the parent class loader.

If there is a security manager, its `checkCreateClassLoader` method is invoked. This may result in a security exception.

Throws:

`SecurityException - If a security manager exists and its checkCreateClassLoader method doesn't allow creation of a new class loader.`

Method Detail

loadClass

```
public Class<?> loadClass(String name)
                   throws ClassNotFoundException
```

Loads the class with the specified binary name. This method searches for classes in the same manner as the `loadClass(String, boolean)` method. It is invoked by the Java virtual machine to resolve class references. Invoking this method is equivalent to invoking `loadClass(name, false)`.

Parameters:

`name - The binary name of the class`

Returns:

`The resulting Class object`

Throws:

`ClassNotFoundException - If the class was not found`

loadClass

```
protected Class<?> loadClass(String name,
                             boolean resolve)
                      throws ClassNotFoundException
```

Loads the class with the specified binary name. The default implementation of this method searches for classes in the following order:

1. Invoke `findLoadedClass(String)` to check if the class has already been loaded.
2. Invoke the `loadClass` method on the parent class loader. If the parent is `null` the class loader built-in to the virtual machine is used, instead.
3. Invoke the `findClass(String)` method to find the class.

If the class was found using the above steps, and the `resolve` flag is true, this method will then invoke the `resolveClass(Class)` method on the resulting `Class` object.

Subclasses of `ClassLoader` are encouraged to override `findClass(String)`, rather than this method.

Unless overridden, this method synchronizes on the result of `getClassLoadingLock` method during the entire class loading process.

Parameters:

`name - The binary name of the class`

resolve - If true then resolve the class

Returns:

The resulting Class object

Throws:

ClassNotFoundException - If the class could not be found

getClassLoadingLock

```
protected Object getClassLoadingLock(String className)
```

Returns the lock object for class loading operations. For backward compatibility, the default implementation of this method behaves as follows. If this ClassLoader object is registered as parallel capable, the method returns a dedicated object associated with the specified class name. Otherwise, the method returns this ClassLoader object.

Parameters:

className - The name of the to-be-loaded class

Returns:

the lock for class loading operations

Throws:

NullPointerException - If registered as parallel capable and className is null

Since:

1.7

See Also:

loadClass(String, boolean)

findClass

```
protected Class<?> findClass(String name)
                      throws ClassNotFoundException
```

Finds the class with the specified binary name. This method should be overridden by class loader implementations that follow the delegation model for loading classes, and will be invoked by the loadClass method after checking the parent class loader for the requested class. The default implementation throws a ClassNotFoundException.

Parameters:

name - The binary name of the class

Returns:

The resulting Class object

Throws:

ClassNotFoundException - If the class could not be found

Since:

1.2

defineClass

```
@Deprecated
protected final Class<?> defineClass(byte[] b,
                                     int off,
                                     int len)
                              throws ClassFormatError
```

Deprecated. *Replaced by* *defineClass(String, byte[], int, int)*

Converts an array of bytes into an instance of class `Class`. Before the `Class` can be used it must be resolved. This method is deprecated in favor of the version that takes a binary name as its first argument, and is more secure.

Parameters:

b - The bytes that make up the class data. The bytes in positions off through off+len-1 should have the format of a valid class file as defined by *The Java™ Virtual Machine Specification*.

off - The start offset in b of the class data

len - The length of the class data

Returns:

The Class object that was created from the specified class data

Throws:

ClassFormatError - If the data did not contain a valid class

IndexOutOfBoundsException - If either off or len is negative, or if off+len is greater than b.length.

SecurityException - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if an attempt is made to define a class in a package with a fully-qualified name that starts with "java.".

See Also:

loadClass(String, boolean), resolveClass(Class)

defineClass

```
protected final Class<?> defineClass(String name,
                                     byte[] b,
```

```
                                        int off,
                                        int len)
                                 throws ClassFormatError
```

Converts an array of bytes into an instance of class `Class`. Before the `Class` can be used it must be resolved.

This method assigns a default `ProtectionDomain` to the newly defined class. The `ProtectionDomain` is effectively granted the same set of permissions returned when `Policy.getPolicy().getPermissions(new CodeSource(null, null))` is invoked. The default domain is created on the first invocation of `defineClass`, and re-used on subsequent invocations.

To assign a specific `ProtectionDomain` to the class, use the `defineClass` method that takes a `ProtectionDomain` as one of its arguments.

Parameters:

`name` - The expected `binary name` of the class, or null if not known

`b` - The bytes that make up the class data. The bytes in positions `off` through `off+len-1` should have the format of a valid class file as defined by *The Java™ Virtual Machine Specification*.

`off` - The start offset in b of the class data

`len` - The length of the class data

Returns:

The Class object that was created from the specified class data.

Throws:

`ClassFormatError` - If the data did not contain a valid class

`IndexOutOfBoundsException` - If either off or len is negative, or if off+len is greater than b.length.

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class (which is unsigned), or if name begins with "java.".

Since:

1.1

See Also:

`loadClass(String, boolean)`, `resolveClass(Class)`, `CodeSource`, `SecureClassLoader`

defineClass

```
protected final Class<?> defineClass(String name,
                                     byte[] b,
                                     int off,
```

```
                                    int len,
                                    ProtectionDomain protectionDomain)
                             throws ClassFormatError
```

Converts an array of bytes into an instance of class `Class`, with an optional `ProtectionDomain`. If the domain is `null`, then a default domain will be assigned to the class as specified in the documentation for `defineClass(String, byte[], int, int)`. Before the class can be used it must be resolved.

The first class defined in a package determines the exact set of certificates that all subsequent classes defined in that package must contain. The set of certificates for a class is obtained from the `CodeSource` within the `ProtectionDomain` of the class. Any classes added to that package must contain the same set of certificates or a `SecurityException` will be thrown. Note that if `name` is `null`, this check is not performed. You should always pass in the binary name of the class you are defining as well as the bytes. This ensures that the class you are defining is indeed the class you think it is.

The specified `name` cannot begin with "`java.`", since all classes in the "`java.*` packages can only be defined by the bootstrap class loader. If `name` is not `null`, it must be equal to the binary name of the class specified by the byte array "`b`", otherwise a `NoClassDefFoundError` will be thrown.

Parameters:

`name` - The expected binary name of the class, or null if not known

`b` - The bytes that make up the class data. The bytes in positions `off` through `off+len-1` should have the format of a valid class file as defined by *The Java™ Virtual Machine Specification*.

`off` - The start offset in `b` of the class data

`len` - The length of the class data

`protectionDomain` - The ProtectionDomain of the class

Returns:

The Class object created from the data, and optional ProtectionDomain.

Throws:

`ClassFormatError` - If the data did not contain a valid class

`NoClassDefFoundError` - If name is not equal to the binary name of the class specified by b

`IndexOutOfBoundsException` - If either off or len is negative, or if off+len is greater than b.length.

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if name begins with "java.".

defineClass

```
protected final Class<?> defineClass(String name,
                                     ByteBuffer b,
                                     ProtectionDomain protectionDomain)
                              throws ClassFormatError
```

Converts a `ByteBuffer` into an instance of class `Class`, with an optional `ProtectionDomain`. If the domain is `null`, then a default domain will be assigned to the class as specified in the documentation for `defineClass(String, byte[], int, int)`. Before the class can be used it must be resolved.

The rules about the first class defined in a package determining the set of certificates for the package, and the restrictions on class names are identical to those specified in the documentation for `defineClass(String, byte[], int, int, ProtectionDomain)`.

An invocation of this method of the form *cl*`.defineClass(`*name*`,` *bBuffer*`,` *pd*`)` yields exactly the same result as the statements

```
...
byte[] temp = new byte[bBuffer.remaining()];
bBuffer.get(temp);
return cl.defineClass(name, temp, 0, temp.length, pd);
```

Parameters:

`name` - The expected `binary name`. of the class, or null if not known

`b` - The bytes that make up the class data. The bytes from positions b.position() through b.position() + b.limit() -1 should have the format of a valid class file as defined by *The Java™ Virtual Machine Specification*.

`protectionDomain` - The ProtectionDomain of the class, or null.

Returns:

The Class object created from the data, and optional ProtectionDomain.

Throws:

`ClassFormatError` - If the data did not contain a valid class.

`NoClassDefFoundError` - If name is not equal to the `binary name` of the class specified by b

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if name begins with "java.".

Since:

1.5

See Also:

defineClass(String, byte[], int, int, ProtectionDomain)

resolveClass

```
protected final void resolveClass(Class<?> c)
```

Links the specified class. This (misleadingly named) method may be used by a class loader to link a class. If the class `c` has already been linked, then this method simply returns. Otherwise, the class is linked as described in the "Execution" chapter of *The Java™ Language Specification*.

Parameters:

`c` - The class to link

Throws:

`NullPointerException` - If `c` is null.

See Also:

defineClass(String, byte[], int, int)

findSystemClass

```
protected final Class<?> findSystemClass(String name)
                                  throws ClassNotFoundException
```

Finds a class with the specified binary name, loading it if necessary.

This method loads the class through the system class loader (see `getSystemClassLoader()`). The `Class` object returned might have more than one `ClassLoader` associated with it. Subclasses of `ClassLoader` need not usually invoke this method, because most class loaders need to override just `findClass(String)`.

Parameters:

`name` - The binary name of the class

Returns:

The `Class` object for the specified `name`

Throws:

`ClassNotFoundException` - If the class could not be found

See Also:

ClassLoader(ClassLoader), getParent()

findLoadedClass

```
protected final Class<?> findLoadedClass(String name)
```

Returns the class with the given binary name if this loader has been recorded by the Java virtual machine as an initiating loader of a class with that binary name. Otherwise `null` is returned.

Parameters:

`name - The binary name of the class`

Returns:

`The Class object, or null if the class has not been loaded`

Since:

`1.1`

setSigners

```
protected final void setSigners(Class<?> c,
                                Object[] signers)
```

Sets the signers of a class. This should be invoked after defining a class.

Parameters:

`c - The Class object`

`signers - The signers for the class`

Since:

`1.1`

getResource

```
public URL getResource(String name)
```

Finds the resource with the given name. A resource is some data (images, audio, text, etc) that can be accessed by class code in a way that is independent of the location of the code.

The name of a resource is a '/'-separated path name that identifies the resource.

This method will first search the parent class loader for the resource; if the parent is `null` the path of the class loader built-in to the virtual machine is searched. That failing, this method will invoke `findResource(String)` to find the resource.

API Note:

When overriding this method it is recommended that an implementation ensures that any delegation is consistent with the getResources(String) method.

Parameters:

name - The resource name

Returns:

A URL object for reading the resource, or null if the resource could not be found or the invoker doesn't have adequate privileges to get the resource.

Since:

1.1

getResources

```
public Enumeration<URL> getResources(String name)
                                throws IOException
```

Finds all the resources with the given name. A resource is some data (images, audio, text, etc) that can be accessed by class code in a way that is independent of the location of the code.

The name of a resource is a /-separated path name that identifies the resource.

The search order is described in the documentation for getResource(String).

API Note:

When overriding this method it is recommended that an implementation ensures that any delegation is consistent with the getResource(String) method. This should ensure that the first element returned by the Enumeration's nextElement method is the same resource that the getResource(String) method would return.

Parameters:

name - The resource name

Returns:

An enumeration of URL objects for the resource. If no resources could be found, the enumeration will be empty. Resources that the class loader doesn't have access to will not be in the enumeration.

Throws:

IOException - If I/O errors occur

Since:

1.2

See Also:

findResources(String)

findResource

```
protected URL findResource(String name)
```

Finds the resource with the given name. Class loader implementations should override this method to specify where to find resources.

Parameters:

name - The resource name

Returns:

A URL object for reading the resource, or null if the resource could not be found

Since:

1.2

findResources

```
protected Enumeration<URL> findResources(String name)
                                  throws IOException
```

Returns an enumeration of URL objects representing all the resources with the given name. Class loader implementations should override this method to specify where to load resources from.

Parameters:

name - The resource name

Returns:

An enumeration of URL objects for the resources

Throws:

IOException - If I/O errors occur

Since:

1.2

registerAsParallelCapable

```
protected static boolean registerAsParallelCapable()
```

Registers the caller as parallel capable. The registration succeeds if and only if all of the following conditions are met:

1. no instance of the caller has been created

2. all of the super classes (except class Object) of the caller are registered as parallel capable

Note that once a class loader is registered as parallel capable, there is no way to change it back.

Returns:

`true if the caller is successfully registered as parallel capable and false if otherwise.`

Since:

`1.7`

getSystemResource

```
public static URL getSystemResource(String name)
```

Find a resource of the specified name from the search path used to load classes. This method locates the resource through the system class loader (see `getSystemClassLoader()`).

Parameters:

`name - The resource name`

Returns:

`A URL object for reading the resource, or null if the resource could not be found`

Since:

`1.1`

getSystemResources

```
public static Enumeration<URL> getSystemResources(String name)
                                                  throws IOException
```

Finds all resources of the specified name from the search path used to load classes. The resources thus found are returned as an `Enumeration` of `URL` objects.

The search order is described in the documentation for `getSystemResource(String)`.

Parameters:

`name - The resource name`

Returns:

`An enumeration of resource URL objects`

Throws:

IOException - If I/O errors occur

Since:

1.2

getResourceAsStream

```
public InputStream getResourceAsStream(String name)
```

Returns an input stream for reading the specified resource.

The search order is described in the documentation for getResource(String).

Parameters:

name - The resource name

Returns:

An input stream for reading the resource, or null if the resource could not be found

Since:

1.1

getSystemResourceAsStream

```
public static InputStream getSystemResourceAsStream(String name)
```

Open for reading, a resource of the specified name from the search path used to load classes. This method locates the resource through the system class loader (see getSystemClassLoader()).

Parameters:

name - The resource name

Returns:

An input stream for reading the resource, or null if the resource could not be found

Since:

1.1

getParent

```
public final ClassLoader getParent()
```

Returns the parent class loader for delegation. Some implementations may use null to represent the bootstrap class loader. This method will return

null in such implementations if this class loader's parent is the bootstrap class loader.

If a security manager is present, and the invoker's class loader is not null and is not an ancestor of this class loader, then this method invokes the security manager's checkPermission method with a RuntimePermission("getClassLoader") permission to verify access to the parent class loader is permitted. If not, a SecurityException will be thrown.

Returns:

The parent ClassLoader

Throws:

SecurityException - If a security manager exists and its checkPermission method doesn't allow access to this class loader's parent class loader.

Since:

1.2

getSystemClassLoader

```
public static ClassLoader getSystemClassLoader()
```

Returns the system class loader for delegation. This is the default delegation parent for new ClassLoader instances, and is typically the class loader used to start the application.

This method is first invoked early in the runtime's startup sequence, at which point it creates the system class loader and sets it as the context class loader of the invoking Thread.

The default system class loader is an implementation-dependent instance of this class.

If the system property "java.system.class.loader" is defined when this method is first invoked then the value of that property is taken to be the name of a class that will be returned as the system class loader. The class is loaded using the default system class loader and must define a public constructor that takes a single parameter of type ClassLoader which is used as the delegation parent. An instance is then created using this constructor with the default system class loader as the parameter. The resulting class loader is defined to be the system class loader.

If a security manager is present, and the invoker's class loader is not null and the invoker's class loader is not the same as or an ancestor of the system class loader, then this method invokes the security manager's checkPermission method with a RuntimePermission("getClassLoader") permission to verify access to the system class loader. If not, a SecurityException will be thrown.

Returns:

The system ClassLoader for delegation, or null if none

Throws:

SecurityException - If a security manager exists and its checkPermission method doesn't allow access to the system class loader.

IllegalStateException - If invoked recursively during the construction of the class loader specified by the "java.system.class.loader" property.

Error - If the system property "java.system.class.loader" is defined but the named class could not be loaded, the provider class does not define the required constructor, or an exception is thrown by that constructor when it is invoked. The underlying cause of the error can be retrieved via the Throwable.getCause() method.

definePackage

```
protected Package definePackage(String name,
                                String specTitle,
                                String specVersion,
                                String specVendor,
                                String implTitle,
                                String implVersion,
                                String implVendor,
                                URL sealBase)
                         throws IllegalArgumentException
```

Defines a package by name in this ClassLoader. This allows class loaders to define the packages for their classes. Packages must be created before the class is defined, and package names must be unique within a class loader and cannot be redefined or changed once created.

Parameters:

name - The package name

specTitle - The specification title

specVersion - The specification version

specVendor - The specification vendor

implTitle - The implementation title

implVersion - The implementation version

implVendor - The implementation vendor

sealBase - If not null, then this package is sealed with respect to the given code source URL object. Otherwise, the package is not sealed.

Returns:

The newly defined Package object

Throws:

IllegalArgumentException - If package name duplicates an existing package either in this class loader or one of its ancestors

Since:

1.2

getPackage

protected Package getPackage(String name)

Returns a Package that has been defined by this class loader or any of its ancestors.

Parameters:

name - The package name

Returns:

The Package corresponding to the given name, or null if not found

Since:

1.2

getPackages

protected Package[] getPackages()

Returns all of the Packages defined by this class loader and its ancestors.

Returns:

The array of Package objects defined by this ClassLoader

Since:

1.2

findLibrary

protected String findLibrary(String libname)

Returns the absolute path name of a native library. The VM invokes this method to locate the native libraries that belong to classes loaded with this

class loader. If this method returns `null`, the VM searches the library along the path specified as the "`java.library.path`" property.

Parameters:

`libname - The library name`

Returns:

`The absolute path of the native library`

Since:

`1.2`

See Also:

`System.loadLibrary(String)`, `System.mapLibraryName(String)`

setDefaultAssertionStatus

```
public void setDefaultAssertionStatus(boolean enabled)
```

Sets the default assertion status for this class loader. This setting determines whether classes loaded by this class loader and initialized in the future will have assertions enabled or disabled by default. This setting may be overridden on a per-package or per-class basis by invoking `setPackageAssertionStatus(String, boolean)` or `setClassAssertionStatus(String, boolean)`.

Parameters:

`enabled - true if classes loaded by this class loader will henceforth have assertions enabled by default, false if they will have assertions disabled by default.`

Since:

`1.4`

setPackageAssertionStatus

```
public void setPackageAssertionStatus(String packageName,
                                      boolean enabled)
```

Sets the package default assertion status for the named package. The package default assertion status determines the assertion status for classes initialized in the future that belong to the named package or any of its "subpackages".

A subpackage of a package named p is any package whose name begins with "`p.`". For example, `javax.swing.text` is a subpackage of `javax.swing`, and both `java.util` and `java.lang.reflect` are subpackages of `java`.

In the event that multiple package defaults apply to a given class, the package default pertaining to the most specific package takes precedence

over the others. For example, if `javax.lang` and `javax.lang.reflect` both have package defaults associated with them, the latter package default applies to classes in `javax.lang.reflect`.

Package defaults take precedence over the class loader's default assertion status, and may be overridden on a per-class basis by invoking `setClassAssertionStatus(String, boolean)`.

Parameters:

`packageName - The name of the package whose package default assertion status is to be set. A null value indicates the unnamed package that is "current" (see section 7.4.2 of *The Java™ Language Specification*.)`

`enabled - true if classes loaded by this classloader and belonging to the named package or any of its subpackages will have assertions enabled by default, false if they will have assertions disabled by default.`

Since:

`1.4`

setClassAssertionStatus

```
public void setClassAssertionStatus(String className,
                                     boolean enabled)
```

Sets the desired assertion status for the named top-level class in this class loader and any nested classes contained therein. This setting takes precedence over the class loader's default assertion status, and over any applicable per-package default. This method has no effect if the named class has already been initialized. (Once a class is initialized, its assertion status cannot change.)

If the named class is not a top-level class, this invocation will have no effect on the actual assertion status of any class.

Parameters:

`className - The fully qualified class name of the top-level class whose assertion status is to be set.`

`enabled - true if the named class is to have assertions enabled when (and if) it is initialized, false if the class is to have assertions disabled.`

Since:

`1.4`

clearAssertionStatus

```
public void clearAssertionStatus()
```

Sets the default assertion status for this class loader to `false` and discards any package defaults or class assertion status settings associated with the class loader. This method is provided so that class loaders can be made to ignore any command line or persistent assertion status settings and "start with a clean slate."

Since:

1.4

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.